# Phase 5 - Incident Response & Recovery

🎯 **Core Requirement:** You must recover the system using stored configuration — NOT by redeploying EC2. This is the skill being tested.

---

## Step 5.1 — Use Logs to Identify Failure Source

```
aws logs filter-log-events \
  --log-group-name /aws/ec2/lab-rds-app \
  --filter-pattern "ERROR" \
  --start-time $(date -d '10 minutes ago' +%s)000 \
  --limit 20
```

### Error Message Decoder

| Error Contains | Root Cause | Fix Path |
|---|---|---|
| "timed out" | Network/SG blocking traffic | Restore SG rule (Scenario A) |
| "Connection refused" | RDS not running | Start RDS instance (Scenario B) |
| "Access denied" | Credential mismatch | Fix Secrets Manager (Scenario C) |
| "Unknown host" | Endpoint/DNS issue | Check Parameter Store endpoint |

---

## Step 5.2 — Retrieve Known-Good Values

### From Parameter Store

```
aws ssm get-parameters \
  --names /lab/db/endpoint /lab/db/port /lab/db/name \
```

```
  --with-decryption \
  --query "Parameters[*].{Name:Name,Value:Value}"
```

### From Secrets Manager

```
aws secretsmanager get-secret-value \
  --secret-id lab/rds/mysql \
  --query "SecretString" \
  --output text | jq .
```

💡 **This is why you stored values in Phase 1.** During an incident, you can retrieve known-good configuration without guessing.

---

## Step 5.3 — Fix Root Cause (Based on Scenario)

### Scenario A Recovery: Restore Security Group

```
aws ec2 authorize-security-group-ingress \
  --group-id <RDS_SG_ID> \
  --protocol tcp \
  --port 3306 \
  --source-group <EC2_SG_ID>
```

### Scenario B Recovery: Start RDS

```
aws rds start-db-instance --db-instance-identifier lab-mysql

# Wait for availability (can take 5-10 minutes)
aws rds wait db-instance-available --db-instance-identifier l
ab-mysql
```

### Scenario C Recovery: Fix Credentials

```
# Update Secrets Manager with CORRECT password
aws secretsmanager update-secret \
  --secret-id lab/rds/mysql \
  --secret-string '{"username":"admin","password":"<CORRECT_P
ASSWORD>","host":"<RDS_ENDPOINT>","port":"3306"}'

# SSH into EC2 and restart app to pick up new credentials
sudo systemctl restart rdsapp
```

---

## Step 5.4 — Verify Recovery WITHOUT Redeploying

```
curl http://<EC2_PUBLIC_IP>/list
```

✅ **Expected:** Application returns data successfully.

---

## Step 5.5 — Verify Alarm Returns to OK

```
aws cloudwatch describe-alarms \
  --alarm-names lab-db-connection-failure-alarm \
  --query "MetricAlarms[0].StateValue"
```

**Expected:** `"OK"` (may take 1-2 minutes to transition)

---

## Common Issues — Phase 5

### ▼ Issue: App still failing after SG/RDS fix

**Symptom:** `curl` still returns error after restoring SG or starting RDS

**Possible Causes:**

1. RDS not fully available yet (Scenario B)
2. App has cached bad state

**Fix:**

```
# Check RDS status
aws rds describe-db-instances \
  --db-instance-identifier lab-mysql \
  --query "DBInstances[0].DBInstanceStatus"

# If needed, restart app (but this is NOT redeploying)
sudo systemctl restart rdsapp
```

## ▼ Issue: App won't recover after credential fix (Scenario C)

**Symptom:** Still getting "Access denied" after updating Secrets Manager

**Root Cause:** App caches credentials at startup

**Fix:** You MUST restart the app:

```
sudo systemctl restart rdsapp
```

> 💡 **Key Insight:** This is exactly what breaks during secret rotation in production. Apps that cache credentials don't pick up rotated secrets until restart.

## ▼ Issue: Alarm stuck in ALARM state

**Symptom:** App recovered but alarm still shows ALARM

**Cause:** CloudWatch evaluates over a period. Wait 1-2 minutes.

**Force check:**

```
aws cloudwatch set-alarm-state \
  --alarm-name lab-db-connection-failure-alarm \
  --state-value OK \
```

```
  --state-reason "Manually reset after recovery verificati
on"
```

## Recovery Success Criteria

🏆 **You have successfully recovered when:**

- ✅ `curl http://<EC2_PUBLIC_IP>/list` ~~returns data~~
- ✅ ~~No new ERROR logs appearing~~
- ✅ ~~Alarm state shows~~ `OK`
- ✅ ~~You did NOT terminate/recreate EC2~~
- ✅ ~~You did NOT run~~ `terraform apply`

## What This Proves

| Action | Skill Demonstrated |
|---|---|
| Read logs to diagnose | Observability literacy |
| Retrieve stored config | Secret management workflow |
| Fix without redeploy | Operational maturity |
| Verify with CLI | Evidence-based recovery |